



Universidade do Minho
Escola de Engenharia

Sistemas Distribuídos

Trabalho Prático

Licenciatura em Engenharia Informática
Ano Letivo 2025/2026

Grupo 18

100751	Diogo Luís Barros Costa
106919	Eduardo Freitas Fernandes
106842	Gonçalo Rodrigues Ribeiro
106888	José Mário Raimundo Lima

Conteúdo

1	Apresentação	1
2	Arquitetura Geral	2
3	Funcionalidades do Serviço	2
3.1	Registo de Eventos	2
3.2	Agregação de Informação	2
3.3	Filtragem de Eventos	3
3.4	Notificação de Ocorrências	3
3.5	Mudança de Dia	3
4	Protocolo de Comunicação	3
4.1	Cliente	3
4.2	Servidor	4
4.2.1	Filtragem de Eventos	5
5	Avaliação Experimental	5
5.1	Cenários de Teste	5
5.2	Análise de Resultados	6
6	Como Executar	7
7	Conclusão	7

1 Apresentação

Neste trabalho prático, desenvolvido no âmbito da unidade curricular de **Sistemas Distribuídos**, tem como principal objetivo a implementação de um **serviço distribuído para o registo e consulta de eventos organizados em séries temporais**. O sistema permite armazenar informação relativa à **venda de produtos** e disponibilizar operações de **agregação, filtragem e notificação de ocorrências** sobre dados históricos.

A solução adotada segue uma arquitetura **cliente-servidor**, na qual um **servidor central** é responsável por gerir a informação e responder a pedidos submetidos por múltiplos clientes através de **sockets TCP**. O sistema foi concebido para funcionar num **ambiente concorrente**, suportando pedidos de vários clientes em simultâneo e permitindo que cada cliente possa efetuar **múltiplos pedidos concorrentes**, sem bloqueios indesejados.

Atendendo ao **potencial elevado volume de dados**, foram implementados mecanismos de **persistência em disco** para armazenar séries temporais de dias anteriores. As operações de agregação são avaliadas de forma *lazy*, sendo os resultados **calculados apenas quando solicitados** e armazenados numa **cache com política LRU**, de modo a limitar o uso de memória e a **melhorar o desempenho global do sistema**.

2 Arquitetura Geral

O código fonte apresentado na diretoria SalesTrace está dividido em três secções: **Servidor**, **Cliente** e **Scripts auxiliares**. Nos ficheiros auxiliares dispomos de uma classe em Java (`Tests.java`) que executa testes de performance sobre o servidor (estes testes são discutidos na secção **Avaliação Experimental**).

Nos módulos de execução do **Cliente**, dispomos de uma interface simples mas funcional sobre a linha de comandos (`SalesTraceClientMain.java`). Esta interface dá uso à Interface `ISalesTraceClient` que por sua vez é implementada pela classe `SalesTraceClient`. Esta biblioteca é multithreaded e disponibiliza as operações do servidor a um cliente.

A secção do **Servidor** é a mais complexa, pois é suportada pelos seguintes módulos:

- `SalesTraceServer`: classe que valida informação e resolve pedidos vindos de clientes
- `ClientHandler`: trata da conexão com um cliente e aplica o protocolo de comunicação estabelecido
- `LastDaysManager`: gestor que disponibiliza operações sobre os dias anteriores
- `ClientsManager`: gestor da informação dos clientes, que regista e valida estes
- `LRUCache`: cache LRU de agregações de informação thread safe.

Existem também certas classes imutáveis que assistem estes módulos, como: `Event`, `CacheKey`, `ClientInfo`, `QueryType` e `RequestType`.

3 Funcionalidades do Serviço

3.1 Registo de Eventos

Uma vez que vários clientes podem adicionar eventos simultaneamente, esta funcionalidade envolve o acesso concorrente a um estado partilhado no servidor. Para garantir consistência dos dados e evitar condições de corrida, a inserção de eventos é realizada dentro de secções críticas protegidas por mecanismos de exclusão mútua, isto é, a série temporal do dia atual (`List<Event>`) é protegida por um `ReentrantLock`.

3.2 Agregação de Informação

Todas as agregações disponibilizadas pelo sistema partilham o mesmo comportamento funcional. Uma thread necessita de analisar uma sequência de dias, logo efetua um pedido ao módulo `LastDaysManager` pela série temporal do dia `x`. Após o tratamento dessa série, a thread indica ao gestor que já não necessita da série através do método `releaseEvents(int d)`. Este mecanismo permite que várias threads consumam a mesma série temporal, mas ao mesmo tempo garante que não existem mais de `S` séries em memória. As séries temporais dos dias passados são imutáveis, logo várias threads podem ler estes recursos de forma concorrente, contribuindo para um grau elevado de paralelismo, aumentando a escalabilidade do sistema. Se já existirem `S` séries em memória, a thread que necessita do recurso terá que entrar numa espera (através de `Condition.await()`) até que exista um lugar livre para a série desejada.

Os resultados das agregações são guardados numa cache LRU, acessível através do nome do produto, número de dias e tipo de agregação. Esta cache é protegida por mecanismos de exclusão mútua, de forma a manter a consistência do sistema.

3.3 Filtragem de Eventos

A funcionalidade de filtragem de eventos distingue-se das restantes funcionalidades do sistema por devolver listas potencialmente extensas de eventos, o que introduz desafios ao nível do volume de dados transmitidos entre cliente e servidor. Esta operação utiliza os mesmos mecanismos que as agregações. A thread atribuída efetua um pedido ao recurso desejado (a série do dia desejado) e filtra os eventos dos produtos. Após o tratamento necessário, indica ao gestor que já não necessita da série. A serialização eficiente está descrita na secção do **Protocolo de Comunicação**.

3.4 Notificação de Ocorrências

A funcionalidade de notificação de ocorrências permite que os clientes sejam informados quando determinados padrões de vendas são verificados, como vendas simultâneas ou consecutivas. Estas operações assumem um comportamento bloqueante, permanecendo ativas até que a condição seja verificada. Esta sincronização é realizada através de mecanismos de espera condicional (`Condition`), permitindo que uma thread suspenda a sua execução até ser explicitamente acordada.

Após um cliente adicionar um evento ao dia corrente, as threads que esperam por notificações são sinalizadas e verificam se a sua condição foi satisfeita. Para a notificação de vendas consecutivas, a thread percorre a série verificando se existem vendas consecutivas de produtos. Para a notificação de vendas simultâneas, a thread percorre a série à procura dos produtos selecionados.

3.5 Mudança de Dia

A mudança de dia é distribuída por três secções do servidor: no módulo dos dia corrente, da cache de agregações e dos dias passados. Esta operação deve ser atômica, não permitindo que outras operações sejam feitas concorrentemente. Através desta análise do facto de esta operação ser pouco frequente, dá-mos uso a um `ReadWriteLock` para esta operação.

Esta operação começa por colocar a série do atual em disco, através de uma serialização binária simples:

- nome do produto (`writeUTF()`)
- quantidade (`writeInt()`)
- preço de venda (`writeFloat()`)

O segundo passo é esvaziar a cache de agregações, e por fim indicar ao gestor dos dias anteriores a existência de um novo dia.

4 Protocolo de Comunicação

Cada mensagem trocada entre servidor e cliente está marcada com um tipo, que permite a ambos identificar o contexto da mensagem. Em cada mensagem é reservado um **byte** para o seu tipo. Na tabela 1 estão apresentados os tipos de mensagens.

4.1 Cliente

Na mensagem de **Autenticação** o cliente envia o nome de utilizador e password. Ambos estes elementos são strings, logo terão comprimentos variáveis, então é necessário indicar o comprimento da string, seguido do seu conteúdo. A função `writeUTF()` é usada para este tipo de serialização.

Tabela 1: Tipos de mensagens

Tipo	Sigla
Autenticação	LOG IN
Registo	SIGN IN
Adicionar Evento	ADD EVENT
Quantidade de Vendas	STOCK
Volume de Vendas	TOTAL
Preço Médio	AVERAGE
Preço Máximo	MAXIMUM
Filtrar Eventos	FILTER
Vendas Simultâneas	SIMULTANEOUS
Vendas Consecutivas	CONSECUTIVE
Mudar de Dia	NEXT DAY
Erro	ERROR

A mensagem é semelhante para o **Registo** de um cliente. O nome de utilizador e palavra passe são enviados, mas também um **booleano** que indica se o cliente é administrador ou não.

Para **Adicionar um Evento** o cliente indica o nome do produto, numa string, a quantidade, num inteiro, e o preço da venda, num float.

No cálculo de **Agregações de Informação**, a mensagem enviada é igual nos 4 tipos (STOCK TOTAL AVERAGE MAXIMUM). O cliente deve enviar o nome do produto, representado numa string, e o número de dias anteriores, num inteiro.

Na **Filtragem de Eventos** o utilizador indica o dia que deseja filtrar (através de um inteiro), o número de produtos que o conjunto de produtos tem (através de um inteiro), e o conjunto do nome dos produtos (através de uma sequência de strings).

Para a notificação de **Vendas Simultâneas** o cliente indica o nome do primeiro produto (numa string) e o nome do segundo produto (também numa string). A mensagem de notificação de **Vendas Consecutivas** é composta apenas pelo número de vendas consecutivas, através de um inteiro.

No pedido de **Mudança de Dia** o cliente não necessita de indicar nenhuma informação extra.

4.2 Servidor

Na resposta às mensagens de **Registo** e **Autenticação**, enviadas pelo cliente, o servidor envia um booleano, que indica se o registo/autenticação teve sucesso, e um inteiro que indica o número de tentativas restantes, caso a operação não tenha sucesso.

Nas operações de **Agregação de Informação** o servidor envia o resultado, no formato de float ou inteiro, consoante o tipo de agregação.

Para a notificação de **Vendas Simultâneas** o servidor envia o resultado, no formato de um booleano, e para a notificação de **Vendas Consecutivas** envia uma flag (no formato de um booleano) que indica se existe resultado ou não, ou seja, se a flag for `true`, é seguida pelo nome do produto (no formato de uma string), se for `false`, significa que houve mudança de dia e o resultado é `null`.

Na operação de **Mudança de Dia** o servidor envia um booleano, que indica se a operação foi efetuada ou não. Esta operação é apenas permitida a administradores, logo o resultado estará dependente do tipo de utilizador.

4.2.1 Filtragem de Eventos

Dado que a lista de eventos devolvida será potencialmente grande, implementamos um método de serialização que compacta a informação desta lista. Os eventos devolvidos são compostos pelos seguintes elementos:

- nome do produto (string)
- quantidade vendida (int)
- preço da venda (float)

Os eventos presentes na lista serão relativos a um conjunto específico de produtos, e dado que o tamanho das strings é variável, serializar estas repetidamente é pouco eficiente, logo decidimos serializar uma vez o nome do produto, dado que é uma componente que se repete várias vezes. O método de serialização é o seguinte:

1. indicar o número de produtos serializados num inteiro
2. indicar o nome do produto numa string
3. indicar o número de eventos (N) relativo a este produto, num inteiro
4. serializar a quantidade vendida, num inteiro
5. serializar o preço da venda, num float
6. repetir 4 e 5 N, vezes
7. voltar a 2

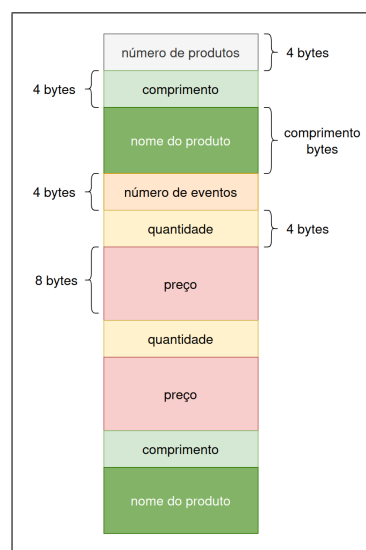


Figura 1: Serialização compacta

5 Avaliação Experimental

A validação experimental da solução proposta foi realizada através de uma bateria de testes automatizada, desenvolvida especificamente para submeter o servidor a cenários de carga intensiva e rigorosa. Esta abordagem permitiu simular um ambiente de alta concorrência, instanciando múltiplos clientes simultâneos para avaliar não só a estabilidade e integridade dos dados sob stress, mas também para aferir métricas quantitativas de desempenho fundamentais, como o débito de processamento e a latência de resposta do serviço.

5.1 Cenários de Teste

Em conformidade com os requisitos do projeto, foram desenhados cenários específicos para avaliar as duas dimensões críticas do sistema - escalabilidade e robustez - focando-se nas funcionalidades nucleares do serviço:

- **Registo de Eventos:** avaliou-se o desempenho da operação de adicionar eventos à série temporal do dia corrente (AddEvent). Este cenário testa a capacidade do servidor em gerir a concorrência na escrita de novos dados em tempo real.

- **Agregação de Informação:** avaliou-se o desempenho das operações de consulta sobre dias anteriores (GetStock, GetProfit, GetAverage, GetMax). Estas operações testam a eficácia do processamento lazy (on demand) e do mecanismo de caching implementado, simulando múltiplos clientes a solicitar agregações sobre subconjuntos de produtos e intervalos de dias.
- **Teste de Robustez:** simulou-se um cenário de falha de consumo por parte de um cliente ("cliente lento"), que estabelece conexão mas não lê as respostas enviadas. O objetivo foi verificar se esta situação bloqueia recursos no servidor, impedindo-o de responder a outros clientes ativos.

5.2 Análise de Resultados

A tabela 2 apresenta os resultados obtidos na execução dos testes nos cenários descritos. A máquina usada para a realização dos testes é a seguinte: ASUS Zenbook, Processador Intel Core i7, 16GiB de Memória, Sistema Operativo Ubuntu 24.04. O servidor recebeu os seguintes **parâmetros**: 5 dias anteriores, 3 séries em memória, 10 entradas na cache.

Tabela 2: Resultados

Operação	Nr Clientes	Nr Operações	Operações/s	Latência (ms)
ADD EVENT	1	1000	23561.51	0.0424
ADD EVENT	10	1000	143693.57	0.0594
ADD EVENT	50	1000	483933.47	0.0827
ADD EVENT	100	1000	741790.46	0.1075
STOCK	1	100	48.24	20.7289
STOCK	10	100	78.85	121.5372
STOCK	50	100	76.65	627.4628
STOCK	100	100	73.85	1297.3056
TOTAL	1	500	53.74	18.6072
TOTAL	10	500	80.97	121.8100
TOTAL	50	500	71.08	687.1654
TOTAL	100	500	70.12	1334.2131
AVERAGE	1	1000	52.77	19.1982
AVERAGE	10	1000	79.07	126.5109
AVERAGE	50	1000	71.60	637.0004
AVERAGE	100	1000	71.03	1204.2000
MAXIMUM	1	50	50.12	18.0072
MAXIMUM	10	100	81.88	120.5690
MAXIMUM	50	500	69.98	651.7004
MAXIMUM	100	1000	72.45	1311.6735

A análise dos dados permite retirar as seguintes conclusões, relacionando-as com as opções de desenho da arquitetura:

1. **Registo de Eventos:** nesta componente verificamos uma diferença perante a agregação de informação, e isso deve-se ao facto de os testes medirem apenas a latência de envio do pedido, e

não da computação efetuada pelo servidor. Isto acontece porque o servidor não envia resposta ao cliente, logo do ponto de vista deste a operação é bastante eficiente.

2. **Agregação de Informação:** neste conjunto de operações podemos verificar que o número de operações efetuadas por segundo estabiliza ao atingir as centenas de clientes. Estes testes mostram que o servidor está sobre alguma carga e que seria necessário uma maior paralelização, como por exemplo na serialização das séries temporais, dado que operações em disco são mais custosas.
3. **Robustez do Serviço:** o teste de robustez foi superado com sucesso ("OK (Passou)"), confirmando que a arquitetura multithreaded isola eficazmente as ligações. O facto de um cliente não consumir as respostas apenas afeta a sua própria conexão (enchendo o buffer TCP), sem degradar o serviço para os restantes utilizadores que continuam a efetuar operações de registo e agregação normalmente.

É de notar que os testes realizados não são suficientes para validar a eficiência total do servidor, pois não são avaliados o consumo de CPU por parte do servidor. O programa não aparenta mostrar uma grande capacidade de escalabilidade, é esperado que aumentando o número de dias, perca performance. Uma alternativa a solucionar este problema seria paralelizar a serialização das séries temporais, pois deverá ser uma das operações mais "pesadas" no servidor.

6 Como Executar

O programa é composto por 4 executáveis: `SalesTraceClientMain.java` que apresenta a interface ao utilizador, `SalesTraceServerMain.java` que inicia o servidor e recebe como parâmetros o número de dias passados, o número máximo de séries em memória, o número máximo de agregações em cache e caminho para a diretoria que contém os dias anteriores em disco. `GenerateDays.java` é um script que gera um número (especificado) de dias anteriores com eventos aleatórios e `Tests.java` é um script que executa os testes, discutidos anteriormente, e recebe como parâmetro o número de dias passados.

Para executar a interface do utilizador e os testes de performance é necessário iniciar o servidor primeiramente. O script de geração de dias é independente dos restantes scripts.

7 Conclusão

Neste trabalho prático foi desenvolvido um serviço distribuído para o registo e consulta de eventos em séries temporais, suportando operações de agregação, filtragem e notificação de ocorrências. A implementação permitiu aplicar conceitos centrais de Sistemas Distribuídos, nomeadamente concorrência, sincronização entre threads e comunicação cliente-servidor.

A arquitetura cliente-servidor, com um servidor multithreaded, permitiu suportar múltiplos clientes concorrentes de forma eficiente. A separação entre dados do dia corrente e dados históricos garantiu consistência e possibilitou operações de leitura paralelas e escaláveis.

Decisões como a avaliação lazy das agregações, a utilização de uma cache LRU e a serialização compacta de dados contribuíram para a redução do custo computacional e da comunicação. As notificações foram implementadas com base em variáveis de condição, permitindo operações bloqueantes sem precisar de uma espera ativa.

Os resultados experimentais obtidos confirmam que o sistema apresenta um comportamento escalável, eficiente e robusto, estando alinhado com os princípios fundamentais de sistemas distribuídos. Apesar dos resultados obtidos, um ponto a melhorar seria a realização de testes mais realistas, de forma a verificar a verdadeira performance do sistema.